

Basic definitions: labelled transition systems

Definition

A *labelled transition system* (LTS) is a tuple

$$\mathcal{T} = (C, \Sigma, \longrightarrow, c_0),$$

where C is a set of configurations, Σ is a finite alphabet, $c_0 \in C$, and

$$\longrightarrow \subseteq C \times \Sigma \times C.$$

We write $c \xrightarrow{a} c'$ for $(c, a, c') \in \longrightarrow$.

An *infinite run* is a sequence

$$c_0 \xrightarrow{a_0} c_1 \xrightarrow{a_1} c_2 \xrightarrow{a_2} \dots,$$

and its *trace* is the word $a_0 a_1 a_2 \dots \in \Sigma^\omega$.

For an operational model M , its *induced LTS* is obtained by taking the configurations of M as vertices and the one-step operational semantics as labelled edges.

Exercise 1: one-counter systems

A *one-counter VASS* consists of a finite set Q of control states and transitions

$$p \xrightarrow{a,d} q, \quad d \in \mathbb{Z}.$$

Its configurations are $(q, n) \in Q \times \mathbb{N}$, and

$$(p, n) \xrightarrow{a} (q, n + d)$$

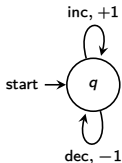
is enabled iff $n + d \geq 0$.

- 1 Consider the one-state system

$$q \xrightarrow{\text{inc}, +1} q, \quad q \xrightarrow{\text{dec}, -1} q.$$

Draw the reachable part of the induced LTS from $(q, 0)$ up to counter value 4.

- 2 Give the same system the $\mathbb{Z}$ -semantics, i.e. let the counter range over $\mathbb{Z}$ and remove the non-negativity test. Draw the part between counter values -3 and 3 .
- 3 Compare the two trace sets. Which traces are removed by the $\mathbb{N}$ -semantics?

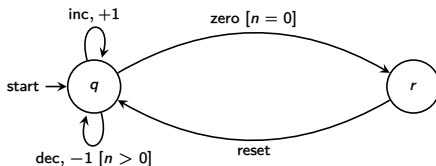


Exercise 2: zero tests

A *one-counter automaton* extends a one-counter VASS by allowing transitions guarded by a test such as $n = 0$ or $n > 0$. Consider:

$$\begin{array}{ll} q \xrightarrow{\text{inc}, +1} q, & q \xrightarrow{\text{dec}, -1} q \quad [n > 0], \\ q \xrightarrow{\text{zero}, 0} r \quad [n = 0], & r \xrightarrow{\text{reset}, 0} q. \end{array}$$

- 1 Draw the part of the induced LTS reachable from $(q, 0)$ with counter values at most 3.
- 2 Mark all configurations from which action zero is enabled.
- 3 Explain why the zero-test transition cannot be simulated by a one-counter VASS transition with the same control graph.



Exercise 3: a pushdown automaton

Definition

A *labelled pushdown automaton* (PDA) is a tuple

$$\mathcal{P} = (Q, \Sigma, \Gamma, \Delta, q_0, \perp),$$

where Q is a finite set of control states, Σ is a finite alphabet, Γ is a finite stack alphabet, $\perp \in \Gamma$ is the bottom symbol, and

$$\Delta \subseteq Q \times \Gamma \times \Sigma \times Q \times \Gamma^{\leq 2}.$$

A configuration is $(q, w) \in Q \times \Gamma^*$, with the top of the stack at the right end. A transition $(p, A, a, q, \beta) \in \Delta$ induces

$$(p, wA) \xrightarrow{a} (q, w\beta).$$

Thus $\beta = \varepsilon$ pops A , $|\beta| = 1$ replaces it, and $|\beta| = 2$ can push one additional symbol. We never pop $\perp$.

Let $Q = \{q\}$ and $\Gamma = \{0, 1, \perp\}$. From every stack content allow push_0 and push_1 ; allow pop whenever the top symbol is not $\perp$.

- 1 Starting from $(q, \perp)$, draw the induced LTS up to stack height 3.
- 2 Ignore edge directions and identify the underlying graph.
- 3 Add a second control state r and transitions that change $q \leftrightarrow r$ without changing the stack. Redraw the first two levels.



Exercise 4: treewidth of a PDA configuration graph

Definition

A *tree decomposition* of an undirected graph $G = (V, E)$ is a tree T whose nodes are labelled by bags $B_t \subseteq V$ such that:

- 1 every vertex of G belongs to some bag;
- 2 for every edge $\{u, v\} \in E$, some bag contains both u and v ;
- 3 for every $v \in V$, the bags containing v form a connected subtree of T .

Its width is $\max_t |B_t| - 1$, and $\text{tw}(G)$ is the minimum possible width.

Assume the pushdown system is in the normal form from Exercise 3.

- 1 Use the tree Γ^* of stack words as the decomposition tree.
- 2 For every $w \in \Gamma^*$ define

$$B_w = (Q \times \{w\}) \cup \bigcup_{A \in \Gamma} (Q \times \{wA\}).$$

Prove that $(B_w)_{w \in \Gamma^*}$ is a tree decomposition of the underlying configuration graph.

- 3 Deduce the bound

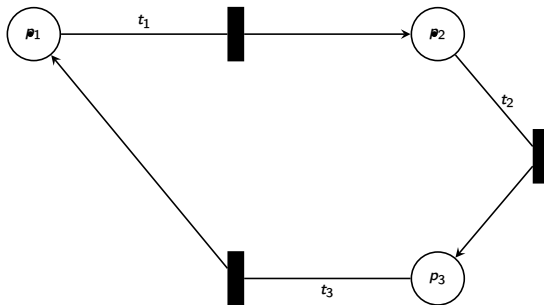
$$\text{tw}(G) \leq |Q|(|\Gamma| + 1) - 1.$$

Courcelle's theorem — version sufficient here

For every fixed k and every fixed MSO sentence φ , model checking φ on finite graphs of treewidth at most k is linear in the size of the graph (given a suitable tree decomposition).

Exercise 5: an S-system

An *S-system* (or *state machine*) is a Petri net in which every transition has exactly one input place and exactly one output place. The marking shown below contains two tokens.

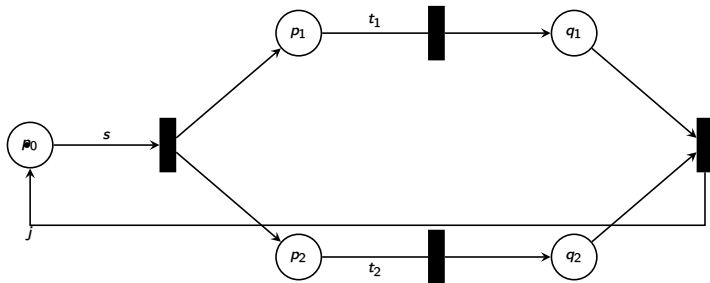


Let a marking be written as $(m(p_1), m(p_2), m(p_3))$.

- 1 List all markings reachable from the initial marking $(1, 1, 0)$.
- 2 Draw the induced LTS. Label every edge by the transition that fires.
- 3 Find an invariant of the net that explains why the reachability set is finite.
- 4 How would the reachable LTS change if the initial marking contained three tokens in p_1 and no tokens elsewhere?

Exercise 6: a T-system

A *T-system* (or *marked graph*) is a Petri net in which every place has exactly one input transition and exactly one output transition. Consider the following marked T-system.

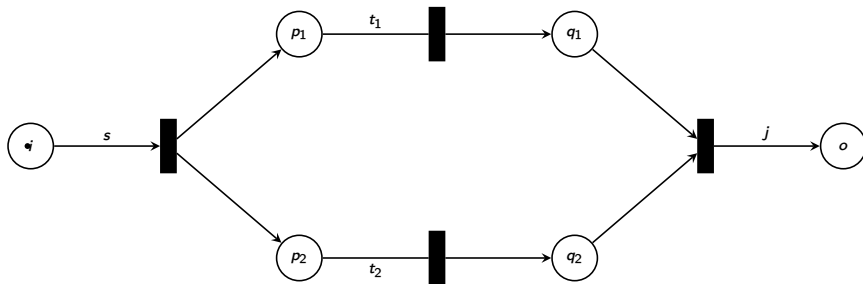


Initially p_0 contains one token and all other places are empty.

- 1 Verify directly from the definition that this net is a T-system.
- 2 List all reachable markings and draw the induced LTS.
- 3 At which marking are t_1 and t_2 concurrently enabled? Show the corresponding diamond in the LTS.
- 4 Is the net bounded? Is it live? Justify both answers from the LTS.

Exercise 7: a workflow net

A *workflow net* has a unique input place i with no incoming arcs, a unique output place o with no outgoing arcs, and every node lies on a directed path from i to o .



The initial marking contains one token in i .

- 1 Verify that the net is a workflow net.
- 2 Draw its complete reachability LTS.
- 3 Show that from every reachable marking it is possible to reach the final marking with one token in o and no token elsewhere.
- 4 Can a token remain anywhere else when o is marked? Is every transition executable in some run? Conclude that this workflow net is sound.
- 5 Add a “short-circuit” transition $o \rightarrow i$. Is the resulting net a T-system?

Büchi automata: definition and warm-up

Definition

A nondeterministic Büchi automaton is

$$A = (Q, \Sigma, \delta, Q_0, F),$$

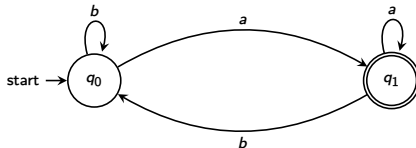
where $\delta \subseteq Q \times \Sigma \times Q$. A run

$$q_0 \xrightarrow{a_0} q_1 \xrightarrow{a_1} q_2 \xrightarrow{a_2} \dots$$

is accepting if it visits F infinitely often. The language $L(A) \subseteq \Sigma^\omega$ consists of words having an accepting run.

Warm-up

Let $\Sigma = \{a, b\}$. Prove that the automaton below accepts exactly the words containing infinitely many occurrences of a .

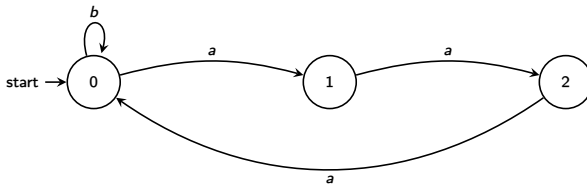


Exercise 8: construct Büchi automata I

Let $\Sigma = \{a, b\}$. Construct a Büchi automaton for each language.

- 1 Words containing infinitely many occurrences of a .
- 2 Words containing only finitely many occurrences of a .
- 3 Words containing infinitely many b such that, between every two *consecutive* b 's, the number of a 's is divisible by 3.
- 4 Words such that, between every two consecutive b 's, the number of a 's is divisible by 3. There is no requirement that b occurs infinitely often.

The following finite-state “modulo 3” component is useful in parts 3–4.



For every construction, state explicitly which states are accepting and explain why the Büchi condition enforces the required infinitary property.

Exercise 9: construct Büchi automata II

Again let $\Sigma = \{a, b\}$.

- 5 The number of b 's is infinite, and only finitely many gaps between consecutive b 's contain a number of a 's divisible by 3.
Equivalently: from some point on, every gap between consecutive b 's has length non-zero modulo 3.
- 6 The number of b 's is finite, and every gap between consecutive b 's contains a number of a 's divisible by 3.
- 7 The number of b 's is finite, and for every two *distinct* occurrences of b , the number of a 's between them is not divisible by 3.

Hint for part 7

Associate with each occurrence of b the number of a 's seen so far, modulo 3. What does the condition say about these residues? In particular, how many occurrences of b can there be?

General design principle

Separate a finite-state *safety condition* (forbidden local patterns) from the Büchi/co-Büchi style requirement saying that something happens infinitely often or only finitely often.

Exercise 10: proving that a language is not Büchi-recognisable

Call $L \subseteq \Sigma^\omega$ ω -regular if it is recognised by a Büchi automaton.
Prove the following two lemmas.

Lemma (ultimately periodic witness)

If $L \neq \emptyset$ is ω -regular, then there exist $u \in \Sigma^*$ and $v \in \Sigma^+$ such that $uv^\omega \in L$.

Lemma (fixed-tail section)

If L is ω -regular and $x \in \Sigma^\omega$ is fixed, then

$$K_x = \{u \in \Sigma^* : ux \in L\}$$

is a regular language of finite words.

Use the lemmas to prove that the following languages are not ω -regular:

1

$$L_1 = \{a b a^2 b a^3 b a^4 b \dots\}.$$

2

$$L_2 = \{a^{n_1} b a^{n_2} b a^{n_3} b \dots : n_1 < n_2 < n_3 < \dots\}.$$

3

Over $\{a, b, c\}$,

$$L_3 = \{a^n b^n c^\omega : n \geq 1\}.$$

For part 3 use the section with tail c^ω and intersect it with $a^+ b^+$.

Exercise 11: Kripke structures as Büchi automata

Definition

A Kripke structure is

$$K = (S, s_0, R, AP, \ell),$$

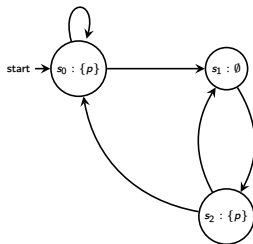
where S is a finite set of states, $R \subseteq S \times S$ is total, AP is a finite set of atomic propositions, and $\ell : S \rightarrow 2^{AP}$ labels states. Its traces are

$$\text{Tr}(K) = \{\ell(s_0)\ell(s_1)\ell(s_2) \cdots : s_0 R s_1 R s_2 R \cdots\} \subseteq (2^{AP})^\omega.$$

- 1 Construct a Büchi automaton A_K such that

$$L(A_K) = \text{Tr}(K).$$

- 2 Your construction may use one fresh initial state and make all states corresponding to S accepting.
- 3 Prove both inclusions.



Exercise 12: intersection with traces — finite case

Let

$$A = (Q_A, \Sigma, \delta, Q_A^0, F)$$

be a Büchi automaton and

$$B = (C, \Sigma, \longrightarrow, c_0)$$

an LTS.

Define their synchronous product. Its configurations are pairs (q, c) and

$$(q, c) \xrightarrow{a} (q', c')$$

iff

$$q \xrightarrow{a}_A q' \quad \text{and} \quad c \xrightarrow{a}_B c'.$$

A product configuration is Büchi-accepting iff $q \in F$.

① Prove:

$$L(A) \cap \text{Tr}(B) \neq \emptyset$$

iff the product has an infinite run visiting $F \times C$ infinitely often.

- ② Assume B is finite. Show that non-emptiness is equivalent to reachability of a strongly connected component that contains an accepting product state and contains a directed cycle.
- ③ Give a graph algorithm for the problem.

Convention

If B is presented as an NFA, use its transition graph as the LTS. The final states of the NFA are irrelevant unless an additional condition explicitly refers to them.

Exercise 13: intersection with traces — infinite-state systems

Let A be a finite Büchi automaton. For each representation of B below, form the synchronous product of A with the induced LTS of B :

- 1 one-counter VASS;
- 2 one-counter automaton;
- 3 d -dimensional VASS;
- 4 pushdown automaton.

For each model:

- (a) Describe a configuration of the product.
- (b) Show that the product remains in the same model class: only the finite control is multiplied by Q_A .
- (c) Reformulate

$$L(A) \cap \text{Tr}(B) = \emptyset$$

as a repeated-reachability/Büchi problem in the product.

- (d) Identify which part of the original infinite-state semantics (counter non-negativity, zero tests, stack operations) is unchanged by the product.

Main observation

A finite specification automaton does not add a new unbounded data structure. It only refines the finite control of the system.

Suggested division into core and optional exercises

Core for a 90-minute tutorial

Exercises 1–2, the drawing part of Exercise 3, Exercises 8–9, Exercise 11, and Exercise 12.

If time permits

Petri-net Exercises 5–7 and the model-class discussion in Exercise 13.

Challenge / homework

The treewidth proof in Exercise 4 and the non- ω -regularity arguments in Exercise 10.